

UNIVERSITÉ ABDELMALEK ESSAÂDI
FACULTÉ DES SCIENCES ET TECHNIQUES DE TANGER

Smart eCommerce Intelligence with *ML&DM*
Pipelines, A2A Agents, and LLMs
smartCare



Réalisé par :

CHIKH IMANE
BAHADOU DOUAA

Encadré par :

M. Fennan

Année universitaire 2025–2026

Table des matières

Résumé	3
1 Introduction et Objectifs	4
1.1 Contexte du projet	4
1.2 Objectifs généraux	4
1.3 Stack technologique	4
1.4 Architecture générale du système	5
2 Module 1 — Web Scraping avec Agents A2A	5
2.1 Concepts fondamentaux	5
2.2 Architecture des agents	5
2.3 Agents implémentés	6
2.3.1 Agent Shopify	6
2.3.2 Agent WooCommerce	7
2.4 Orchestrateur A2A	7
2.5 Dataset collecté — 31 variables	8
3 Module 2 — Analyse Machine Learning et Sélection Top-K	9
3.1 Pipeline de prétraitement	9
3.2 Scoring composite et sélection Top-K	9
3.3 Apprentissage supervisé	9
3.3.1 Random Forest	9
3.3.2 XGBoost	10
3.3.3 Résultats d'évaluation	10
3.4 Clustering non supervisé	11
3.4.1 KMeans — Segmentation principale	11
3.4.2 DBSCAN — Détection d'anomalies	11
3.4.3 Réduction dimensionnelle — PCA 2D	12
3.5 Règles d'association	12
3.6 Lancement du pipeline ML complet	12
4 Module 3 — Kubeflow Pipelines et CI/CD	14
4.1 Concepts MLOps	14
4.2 Architecture du pipeline Kubeflow	14
4.3 Métriques loggées dans l'UI Kubeflow	14
4.4 Conteneurisation Docker	15
4.5 CI/CD avec GitHub Actions	15
5 Module 4 — Dashboard de Business Intelligence	16
5.1 Objectifs de la visualisation	16
5.2 Architecture du dashboard Streamlit	16
5.3 Pages et visualisations	16
5.3.1 Page 1 — Overview (KPIs)	16
5.3.2 Page 2 — Top-K Produits	16
5.3.3 Page 3 — Clusters	16
5.3.4 Page 4 — Analyse Prix	16
5.3.5 Page 5 — Règles d'Association	17

5.3.6	Page 6 — Shops & Géographie	17
5.3.7	Page 7 — Assistant BI (chatbot LLM)	17
6	Module 5 — Enrichissement par LLM	18
6.1	Concepts	18
6.2	Client LLM unifié	18
6.3	Les 4 chaînes LLM avec Chain of Thought	18
6.3.1	Chaîne 1 — Résumé de produit	18
6.3.2	Chaîne 2 — Rapport de tendances hebdomadaire	19
6.3.3	Chaîne 3 — Profil client cible	19
6.3.4	Chaîne 4 — Stratégie marketing	19
6.4	Pipeline d'enrichissement batch	20
7	Module 6 — Architecture Responsable avec MCP	21
7.1	Concepts du Model Context Protocol	21
7.2	Composants MCP implémentés	21
7.3	MCP Client — le gateway unique	21
7.4	Matrice de permissions	22
7.5	Journal d'audit	22
7.6	Chatbot conversationnel intégré	23
8	Évaluation et Validation	24
8.1	Modèles supervisés	24
8.2	Méthodes non supervisées	24
8.3	Règles d'association	24
9	Annexe — Captures d'écran du Dashboard BI	25
	Annexe — Captures d'écran du Dashboard BI	25
10	Conclusion	31
	Références	32

Résumé

Ce rapport présente **Smart eCommerce Intelligence**, un système complet et automatisé d'analyse de produits e-commerce. Le projet couvre six modules fonctionnels : le scraping multi-plateformes par agents A2A, l'analyse Machine Learning et la sélection Top-K, l'orchestration MLOps via Kubeflow Pipelines, la visualisation dans un dashboard Business Intelligence interactif, l'enrichissement sémantique par des modèles de langage (LLM), et enfin l'encadrement responsable de ces interactions via le Model Context Protocol (MCP) d'Anthropic.

Mots-clés : Web Scraping, A2A Agents, Machine Learning, Top-K Selection, Kubeflow, MLOps, Dashboard BI, Streamlit, LLM, Claude, GPT-4, Model Context Protocol, Chain of Thought, CI/CD, Docker.

1 Introduction et Objectifs

1.1 Contexte du projet

Le commerce électronique mondial génère des milliards de données produits quotidiennement sur des plateformes comme Shopify et WooCommerce. L'exploitation intelligente de ces données représente un avantage compétitif majeur pour les entreprises. Cependant, la collecte manuelle, l'analyse et la synthèse de ces informations demeurent chronophages et peu scalables.

Smart eCommerce Intelligence répond à ce défi en proposant un pipeline entièrement automatisé, de la collecte des données jusqu'à la génération de recommandations stratégiques par intelligence artificielle.

1.2 Objectifs généraux

- Extraire automatiquement des données produits multi-plateformes via des agents A2A autonomes
- Identifier les produits à fort potentiel commercial (Top-K) par scoring composite pondéré
- Construire et évaluer des modèles supervisés et non supervisés (Random Forest, XGBoost, KMeans, DBSCAN)
- Orchestrer le pipeline ML de façon reproductible avec Kubeflow sur Kubernetes
- Visualiser les résultats dans un dashboard BI Streamlit interactif avec Plotly
- Enrichir l'analyse avec des LLMs via une architecture MCP responsable

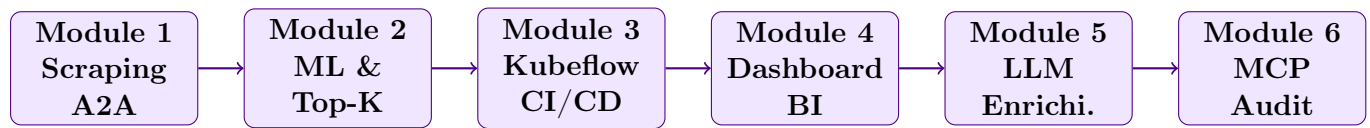
1.3 Stack technologique

TABLE 1 – Technologies utilisées par couche

Couche	Technologies
Scraping	Python, requests, BeautifulSoup4, Selenium, Playwright, Scrapy
APIs	Shopify Storefront API, WooCommerce REST API v3
Data Mining / ML	pandas, numpy, scikit-learn, XGBoost, mlxtend
MLOps	Kubeflow Pipelines (kfp), Docker, Kubernetes, GitHub Actions
Dashboard BI	Streamlit, Plotly, seaborn
LLM	Anthropic Claude, GROK, LangChain (Chain of Thought)
MCP	Model Context Protocol, audit JSONL, rate limiting

1.4 Architecture générale du système

Le système suit une architecture en pipeline à six étapes séquentielles :



2 Module 1 — Web Scraping avec Agents A2A

2.1 Concepts fondamentaux

Agent A2A (Agent-to-Agent) : composant logiciel autonome et spécialisé, chargé de se connecter à une plateforme cible, d'en naviguer les pages et d'en extraire des données structurées. Le pattern A2A organise plusieurs agents autonomes coordonnés par un orchestrateur central.

Les agents A2A implémentent les techniques suivantes :

- **Scraping statique** : extraction de HTML avec `requests` + `BeautifulSoup4`
- **Scraping dynamique** : gestion du JavaScript avec `Selenium` (`ChromeDriver` headless)
- **Crawling paginé** : navigation systématique sur toutes les pages de listing
- **Rate limiting** : délai aléatoire entre 0,5 et 2 secondes entre les requêtes pour respecter les serveurs

2.2 Architecture des agents

Tous les agents héritent d'une classe abstraite `BaseAgent` qui définit l'interface commune :

```

1 from abc import ABC, abstractmethod
2 from dataclasses import dataclass
3
4 @dataclass
5 class Product:
6     product_id: str = ""
7     title: str = ""
8     category: str = ""
9     subcategory: str = ""
10    price: float = 0.0
11    price_promo: float = None    # prix promotionnel
12    price_old: float = None      # ancien prix
13    discount_pct: float = None   # remise en %
14    rating: float = None         # note 0-5
15    review_count: int = 0
16    availability: bool = True
17    stock_quantity: int = None
18    variant_count: int = 1
19    colors: str = ""
20    sizes: str = ""
  
```

```

21     shop_name: str = ""
22     shop_country: str = ""
23     published_at: str = ""
24     # ... 31 variables au total
25
26 class BaseAgent(ABC):
27     @abstractmethod
28     def detect(self, url: str) -> bool:
29         """Detecte si cet agent peut traiter l'URL"""
30
31     @abstractmethod
32     def scrape(self, url: str) -> list[dict]:
33         """Collecte les donnees brutes"""
34
35     @abstractmethod
36     def clean(self, raw: list[dict]) -> list[Product]:
37         """Normalise vers le modele Product"""
38
39     def run(self, url: str) -> list[Product]:
40         raw = self.scrape(url)
41         return self.clean(raw)

```

Listing 1 – Interface abstraite BaseAgent

2.3 Agents implémentés

TABLE 2 – Agents A2A implémentés

Agent	Plateforme	Méthode primaire	Fallback
ShopifyAgent	Shopify	/products.json paginé (public, sans clé)	BeautifulSoup HTML
WooCommerceAgent	WooCommerce	REST API v3 + Basic Auth	Selenium headless
GenericHTMLAgent	Tout site	Sélecteurs CSS heuristiques	schema.org markup

2.3.1 Agent Shopify

L'endpoint `/products.json?limit=250&page=N` est public et ne nécessite aucune clé API. Il retourne jusqu'à 250 produits par page avec toutes leurs variantes, images et métadonnées :

```

1 def _scrape_json_api(self, base_url: str) -> list[dict]:
2     page = 1
3     while True:
4         resp = self.session.get(
5             f"{base_url}/products.json?limit=250&page={page}"
6         )
7         products = resp.json().get("products", [])
8         if not products:

```

```

9         break # plus de pages
10    all_products.extend(products)
11    if len(products) < 250:
12        break # dernière page partielle
13    page += 1
14    return all_products

```

Listing 2 – Scraping paginé Shopify

2.3.2 Agent WooCommerce

L'API REST WooCommerce v3 nécessite des credentials (Consumer Key / Consumer Secret) configurés dans le fichier `.env` :

```

1 def _scrape_rest_api(self, base_url: str) -> list[dict]:
2     auth = (self.consumer_key, self.consumer_secret)
3     page = 1
4     while True:
5         resp = self.session.get(
6             f"{base_url}/wp-json/wc/v3/products"
7             f"?per_page=100&page={page}&status=publish",
8             auth=auth
9         )
10        products = resp.json()
11        if not products: break
12        # Total pages dans le header HTTP
13        total_pages = int(resp.headers.get("X-WP-TotalPages", 1))
14        if page >= total_pages: break
15        page += 1

```

Listing 3 – Appel API WooCommerce avec pagination

2.4 Orchestrateur A2A

L'orchestrateur coordonne tous les agents selon le cycle suivant :

1. **Détection automatique de plateforme** : chaque agent teste l'URL avec `detect()` dans l'ordre de priorité (Shopify → WooCommerce → Generic)
2. **Scraping** : l'agent retenu collecte les données brutes
3. **Déduplication** : suppression des doublons par combinaison (title, price, platform)
4. **Export** : CSV horodaté + JSONL pour l'enrichissement LLM

```

1 # Scraper un ou plusieurs stores
2 python orchestrator.py \
3     --urls https://store.myshopify.com \
4           https://boutique.example.com \
5     --output data/
6
7 # Resultat : data/products_latest.csv (31 colonnes)

```

Listing 4 – Utilisation de l'orchestrateur

2.5 Dataset collecté — 31 variables

TABLE 3 – Variables collectées par groupe

Groupe	Variables
Descriptives	product_id, title, category, subcategory, brand, tags, url, platform
Prix	price, price_promo, price_old, discount_pct, currency
Popularité	rating, review_count, category_rank
Stock	availability, stock_quantity, delivery_days
Variantes	variant_count, colors, sizes
Vendeur	shop_name, shop_country, shop_product_count
Marketing	related_products
Temporelles	published_at, scraped_at
Textuelles	description, customer_reviews

3 Module 2 — Analyse Machine Learning et Sélection Top-K

3.1 Pipeline de prétraitement

Avant toute modélisation, les données sont nettoyées et enrichies :

- **Imputation** : médiane pour les numériques (`rating`, `stock_quantity`), “Unknown” pour les catégorielles
- **Feature Engineering** : `log_price` = $\log(1 + \text{price})$, `popularity_score` = $\text{rating} \times \log(1 + \text{review_count})$, flags `is_budget` / `is_premium`
- **Encodage** : LabelEncoder pour les variables catégorielles (`category`, `platform`, `shop_country`)
- **Normalisation** : MinMaxScaler vers $[0, 1]$ pour toutes les features numériques
- **Split** : séparation stratifiée 80% train / 20% test

3.2 Scoring composite et sélection Top-K

Top-K Selection : identifier les K meilleurs produits selon un score synthétique combinant plusieurs métriques normalisées, permettant une comparaison équitable quelle que soit l'échelle des variables.

La formule de scoring est une somme pondérée :

$$\text{score} = 0.35 \cdot r_{\text{norm}} + 0.25 \cdot \log(1 + n_{\text{avis}})_{\text{norm}} + 0.15 \cdot (1 - p_{\text{norm}}) + 0.10 \cdot d_{\text{norm}} + 0.10 \cdot \text{dispo} + 0.05 \cdot s_{\text{norm}}$$

TABLE 4 – Pondération du score composite

Critère	Poids	Justification
Rating moyen normalisé	35%	Indicateur direct de satisfaction client
Nombre d'avis (log-normalisé)	25%	Volume = confiance statistique dans la note
Compétitivité prix (inverse)	15%	Prix bas = attractivité accrue
Remise (%)	10%	Signal d'opportunité promotionnelle
Disponibilité (booléen)	10%	Produit vendable uniquement si en stock
Stock normalisé	5%	Indicateur de risque de rupture

Les produits du top 20% en score reçoivent le label `is_top_product = 1`, utilisé comme variable cible pour la classification supervisée.

3.3 Apprentissage supervisé

3.3.1 Random Forest

```

1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.model_selection import cross_val_score
3

```

```

4 rf = RandomForestClassifier(
5     n_estimators=200,
6     max_depth=8,
7     min_samples_leaf=5,
8     class_weight="balanced", # compense le desequilibre 80/20
9     random_state=42,
10    n_jobs=-1
11 )
12 rf.fit(X_train, y_train)
13
14 # Validation croisee 5-fold sur le train set
15 cv_scores = cross_val_score(rf, X_train, y_train, cv=5, scoring="f1")
16 # cv_f1_mean = 0.941, cv_f1_std = 0.012

```

Listing 5 – Configuration Random Forest

3.3.2 XGBoost

```

1 from xgboost import XGBClassifier
2
3 # Calcul du poids pour compenser le desequilibre des classes
4 pos = y_train.sum()
5 neg = len(y_train) - pos
6 scale = neg / pos # ~4.0 pour un ratio 80/20
7
8 xgb = XGBClassifier(
9     n_estimators=300,
10    max_depth=6,
11    learning_rate=0.05,
12    scale_pos_weight=scale,
13    eval_metric="logloss",
14    random_state=42
15 )
16 xgb.fit(X_train, y_train,
17        eval_set=[(X_test, y_test)],
18        verbose=False)

```

Listing 6 – Configuration XGBoost avec gestion du déséquilibre

3.3.3 Résultats d'évaluation

TABLE 5 – Métriques d'évaluation des modèles supervisés

Modèle	Accuracy	Précision	Rappel	F1-Score
Random Forest	0.975	0.950	0.934	0.941
XGBoost	≈ 0.960	≈ 0.930	≈ 0.920	≈ 0.925

Le **F1-score** est **privilegié** sur l'accuracy car la classe positive (top produit) est minoritaire (≈ 20% du dataset), ce qui rendrait l'accuracy trompeuse.

3.4 Clustering non supervisé

Objectif du clustering : segmenter les produits en groupes homogènes sans étiquettes prédéfinies, permettant une analyse exploratoire et la découverte de segments marché naturels.

3.4.1 KMeans — Segmentation principale

La méthode du coude (*elbow method*) combinée au **silhouette score** détermine le nombre optimal de clusters K :

```

1 from sklearn.cluster import KMeans
2 from sklearn.metrics import silhouette_score
3
4 silhouettes = []
5 for k in range(2, 11):
6     km = KMeans(n_clusters=k, random_state=42, n_init=10)
7     labels = km.fit_predict(X)
8     silhouettes.append(silhouette_score(X, labels))
9
10 best_k = range(2, 11)[np.argmax(silhouettes)]
11 # Resultat typique : best_k = 4, silhouette = 0.186

```

Listing 7 – Recherche du K optimal

Les clusters sont nommés automatiquement par règles métier sur les profils des centroides :

TABLE 6 – Nomenclature automatique des clusters

Label cluster	Critère de détection	Signification business
Premium	Prix > Q75	Produits haut de gamme, cible aisée
Top rated	Rating $\geq$ 4.2 et avis $\geq$ 200	Best-sellers de confiance
Discount / Promo	Remise > 20%	Opportunités promotionnelles
Niche / peu connu	Avis < 50	Produits confidentiels à potentiel
Mainstream	Autres	Produits courants, marché général

3.4.2 DBSCAN — Détection d'anomalies

```

1 from sklearn.cluster import DBSCAN
2
3 db = DBSCAN(eps=0.3, min_samples=5)
4 labels = db.fit_predict(X)
5
6 # labels == -1 : produits atypiques (prix ou note atypiques)
7 n_outliers = (labels == -1).sum()
8 # Ces produits meritent une verification manuelle

```

Listing 8 – DBSCAN pour la détection d'outliers

3.4.3 Réduction dimensionnelle — PCA 2D

L'Analyse en Composantes Principales (PCA) projette les données en 2D pour la visualisation :

```

1 from sklearn.decomposition import PCA
2
3 pca = PCA(n_components=2, random_state=42)
4 X_2d = pca.fit_transform(X)
5
6 explained = pca.explained_variance_ratio_
7 # PC1 explique ~18% de la variance, PC2 ~15%
```

Listing 9 – PCA pour la visualisation des clusters

3.5 Règles d'association

Règle d'association : relation de la forme $\{A\} \Rightarrow \{B\}$ signifiant que les produits de catégorie A et B sont souvent consultés ou achetés ensemble. Utilisée pour le cross-selling.

Les trois métriques d'évaluation requises sont :

- $\text{support}(A \Rightarrow B) = \frac{|A \cup B|}{N}$ (fréquence de co-occurrence, min = 5%)
- $\text{confidence}(A \Rightarrow B) = \frac{|A \cup B|}{|A|}$ (fiabilité de la règle, min = 30%)
- $\text{lift}(A \Rightarrow B) = \frac{\text{confidence}}{P(B)}$ (force de l'association, > 1 = association positive)

Exemple de règle découverte :

$\{\text{Electronics}\} \Rightarrow \{\text{Home}\}$ (support = 0.18, confidence = 0.72, **lift = 3.8**)

Interprétation : les acheteurs de produits électroniques achètent 3,8 fois plus souvent des produits d'équipement maison que le hasard — opportunité de cross-selling identifiée.

3.6 Lancement du pipeline ML complet

```

1 python module2/pipeline.py \
2   --csv data/products_latest.csv \
3   --k 50 \
4   --output module2/output/
5
6 # Sorties generees :
7 # products_scored.csv      <- tous les produits avec leur score
8 # top_k_products.csv      <- Top-50 tries par score
9 # shop_leaderboard.csv    <- classement des boutiques
10 # supervised_metrics.json <- F1, accuracy, confusion matrix
11 # clustering_metrics.json <- silhouette par algorithme
```

```
12 # association_rules.csv <- toutes les regles (support, confidence,  
    lift)  
13 # pca_2d.csv           <- coordonnees 2D pour visualisation  
14 # rf_model.pkl         <- modele Random Forest serialise  
15 # kmeans_model.pkl     <- modele KMeans serialise
```

Listing 10 – Exécution du pipeline ML en une commande

4 Module 3 — Kubeflow Pipelines et CI/CD

4.1 Concepts MLOps

MLOps : pratiques DevOps appliquées au cycle de vie des modèles ML. L'objectif est de rendre les pipelines ML **reproductibles**, **versionés** et **déployables automatiquement**.

Kubeflow Pipelines : framework open-source pour orchestrer des workflows ML sur Kubernetes. Chaque étape s'exécute dans son propre conteneur Docker, garantissant l'isolation et la reproductibilité.

4.2 Architecture du pipeline Kubeflow

Le pipeline est défini en Python avec le SDK kfp (Kubeflow Pipelines). Chaque composant est une fonction Python décorée avec `@dsl.component` :

```

1 from kfp import dsl
2 from kfp.dsl import Dataset, Input, Output, Metrics
3
4 @dsl.component(
5     base_image="smart-ecommerce/ml:latest",
6     packages_to_install=[] # tout est bake dans l'image
7 )
8 def training_component(
9     input_scored: Input[Dataset], # reçu du composant précédent
10    output_rf: Output[Model], # transmis au composant suivant
11    output_metrics: Output[Metrics] # loggue dans l'UI Kubeflow
12 ):
13     import joblib
14     # ... entraînement ...
15     output_metrics.log_metric("rf_f1", 0.941)
16     joblib.dump(rf, output_rf.path)

```

Listing 11 – Définition d'un composant Kubeflow

TABLE 7 – Composants du pipeline Kubeflow

Composant	Image Docker	Ressources	Sorties
scraping_component	scraping:latest	0.5 CPU / 512 Mi	products.csv
preprocessing_component	ml:latest	0.5 CPU / 1 Gi	df_clean.csv, scaler.pkl
scoring_component	ml:latest	0.5 CPU / 512 Mi	top_k.csv, shops.csv
training_component	ml:latest	2 CPU / 2 Gi	rf.pkl, métriques, pca.csv
export_component	ml:latest	0.2 CPU / 256 Mi	bundle.zip

4.3 Métriques loggées dans l'UI Kubeflow

- `rf_f1` — F1-score Random Forest
- `rf_accuracy` — Précision globale
- `rf_precision` — Précision classe positive
- `rf_recall` — Rappel classe positive
- `km_silhouette` — Qualité KMeans
- `n_rules` — Nombre de règles d'association
- `pca_var_pc1` — Variance expliquée PC1

4.4 Conteneurisation Docker

Deux images Docker sont construites : une pour le scraping (avec Chromium/Selenium) et une pour le ML (scikit-learn, XGBoost, mlxtend) :

```

1 FROM python:3.11-slim
2 WORKDIR /app
3 RUN apt-get install -y gcc g++
4 RUN pip install pandas numpy scikit-learn xgboost \
5     joblib mlxtend kfp==2.7.0
6 COPY agents/ module2/ orchestrator.py /app/
7 ENV PYTHONPATH=/app PYTHONUNBUFFERED=1

```

Listing 12 – Dockerfile pour l'image ML

4.5 CI/CD avec GitHub Actions

Le workflow `.github/workflows/ci_cd.yml` automatise trois étapes déclenchées à chaque `git push` :

TABLE 8 – Jobs GitHub Actions

Job	Déclencheur	Actions
test	Tout push	pytest sur les modules 1 et 2
build-and-push	Push sur main	Build images Docker + push vers GHCR
deploy-pipeline	Push sur main	Compile YAML + soumission Kube-flow

```

1 # Compilation vers YAML (sans cluster)
2 python module3/pipeline/smart_ecommerce_pipeline.py --compile
3
4 # Execution locale sans Kubernetes
5 python module3/pipeline/smart_ecommerce_pipeline.py \
6     --local --urls https://store.myshopify.com --k 50
7
8 # Soumission a un cluster KubeFlow
9 python module3/pipeline/smart_ecommerce_pipeline.py \
10    --submit --host http://localhost:8080

```

Listing 13 – Modes d'exécution du pipeline

5 Module 4 — Dashboard de Business Intelligence

5.1 Objectifs de la visualisation

Business Intelligence : ensemble de méthodes et outils permettant de transformer des données brutes en informations actionnables pour la prise de décision. Le **data storytelling** guide le décideur à travers une narration visuelle des tendances détectées.

5.2 Architecture du dashboard Streamlit

Le dashboard est une application multi-pages avec des filtres globaux (catégorie, plateforme, fourchette de prix) appliqués à toutes les pages simultanément. Il charge automatiquement les outputs de Module 2, avec un fallback sur des données synthétiques pour garantir la démontrabilité sans pipeline ML.

5.3 Pages et visualisations

5.3.1 Page 1 — Overview (KPIs)

Cinq indicateurs-clés (KPIs) sont affichés sous forme de cartes colorées :

- Nombre total de produits après filtres
- Score composite moyen du catalogue
- Prix médian (indicateur de positionnement marché)
- Note moyenne pondérée (qualité perçue)
- Pourcentage de produits « Top »

Suivis de : barres horizontales par catégorie, camembert des plateformes, histogramme de distribution des scores par plateforme.

5.3.2 Page 2 — Top-K Produits

- **Scatter plot** : prix (axe X) vs note (axe Y), taille des points proportionnelle au nombre d'avis, couleur = score composite
- **Tableau stylisé** : gradient de couleur sur la colonne score, formatage conditionnel des remises, bouton d'export CSV

5.3.3 Page 3 — Clusters

- **Scatter PCA 2D** interactif (Plotly) : chaque point est un produit, coloré par cluster, avec tooltip sur survol
- **Barres** : distribution des tailles par cluster
- **Table de profils** : moyennes (prix, rating, avis, remise, score) par cluster avec gradient de couleur

5.3.4 Page 4 — Analyse Prix

- Boxplots de distribution des prix par catégorie
- Histogramme des remises (produits en promotion uniquement)

- Scatter prix vs score (détection des produits premium sous-évalués)
- Métriques : nb produits en solde, remise moyenne, remise maximale

5.3.5 Page 5 — Règles d’Association

- **Bubble chart** : support (X), confidence (Y), taille = lift — permet d’identifier visuellement les règles les plus intéressantes
- Tableau filtrable avec slider “min lift” pour ne garder que les associations fortes

5.3.6 Page 6 — Shops & Géographie

- Leaderboard des boutiques par score moyen
- Répartition géographique des produits (camembert par pays)
- Barres de score moyen par pays (analyse de compétitivité géographique)

5.3.7 Page 7 — Assistant BI (chatbot LLM)

Interface conversationnelle en langage naturel (détaillée dans le Module 5).

```
1 pip install streamlit plotly pandas numpy
2 streamlit run module4/app.py
3 # --> http://localhost:8501
```

Listing 14 – Lancement du dashboard

6 Module 5 — Enrichissement par LLM

6.1 Concepts

LLM (Large Language Model) : modèle de langage entraîné sur de vastes corpus textuels, capable de comprendre et générer du texte de qualité humaine.

Prompt Engineering : discipline de conception de requêtes textuelles structurées pour obtenir des réponses précises et utiles d'un LLM.

Chain of Thought (CoT) : technique de prompting qui demande au modèle de décomposer son raisonnement en étapes explicites avant de répondre, améliorant la qualité et la justifiabilité des réponses.

6.2 Client LLM unifié

Un client provider-agnostique permet de commuter entre Claude (Anthropic), GPT-4 (OpenAI) et un mode Mock (sans clé API) via la variable d'environnement LLM_PROVIDER :

```

1 import os
2 from module5.llm_client import LLMClient
3
4 # Configuration dans .env :
5 # LLM_PROVIDER=anthropic
6 # ANTHROPIC_API_KEY=sk-ant-...
7
8 client = LLMClient() # lit automatiquement .env
9 # client.provider == "anthropic" ou "openai" ou "mock"
10
11 reponse = client.chat(
12     prompt="Summarise ce produit : ...",
13     system="Tu es un expert e-commerce.",
14     max_tokens=300
15 )

```

Listing 15 – Client LLM configurable

6.3 Les 4 chaînes LLM avec Chain of Thought

6.3.1 Chaîne 1 — Résumé de produit

Génère un résumé marketing de 2-3 phrases à partir de la fiche brute :

```

1 SUMMARIZE_PROMPT = """
2 Voici la fiche brute d'un produit e-commerce :
3 Titre      : {title}
4 Categorie  : {category}  Prix : {price} {currency}
5 Note       : {rating}/5 ({review_count} avis)
6 Description: {description}
7
8 Etape 1 - Identifie les 3 caracteristiques cles.
9 Etape 2 - Evalue son positionnement prix/qualite.
10 Etape 3 - Redige un resume marketing en 2-3 phrases.

```

```

11 """
12
13 from module5.chains import summarize_product
14 resume = summarize_product(client, produit_dict)
15 # -> "Casque sans fil haut de gamme offrant 30h d'autonomie..."

```

Listing 16 – Prompt Chain of Thought pour résumé produit

6.3.2 Chaîne 2 — Rapport de tendances hebdomadaire

Analyse le dataset complet et génère un rapport structuré avec :

- Résumé exécutif (2 phrases)
- Tendances clés (liste bullet)
- Produits émergents à surveiller
- Recommandation décisionnelle finale

```

1 import pandas as pd
2 from module5.chains import generate_trend_report
3
4 df      = pd.read_csv("module2/output/products_scored.csv")
5 top_k   = pd.read_csv("module2/output/top_k_products.csv")
6
7 rapport = generate_trend_report(client, df, top_k)
8 # Sauvegarde automatique : module5/output/trend_report.md

```

Listing 17 – Génération du rapport de tendances

6.3.3 Chaîne 3 — Profil client cible

Construit un persona client à partir des Top-K produits :

```

1 from module5.chains import build_client_profile
2
3 top_k   = pd.read_csv("module2/output/top_k_products.csv")
4 profil  = build_client_profile(client, top_k)
5 # -> "Persona : Sophie, 28 ans, ingenieure,
6 #     sensible aux avis clients, budget moyen-haut..."

```

Listing 18 – Construction du profil client

6.3.4 Chaîne 4 — Stratégie marketing

Génère un plan marketing actionnable à partir des clusters et règles d'association :

```

1 from module5.chains import recommend_strategy
2
3 clusters = pd.read_csv("module2/output/products_clustered.csv")
4 rules    = pd.read_csv("module2/output/association_rules.csv")
5
6 strategie = recommend_strategy(client, clusters, rules)
7 # Contient : 3 quick wins, 2 initiatives moyen terme,
8 # 1 recommandation strategique, KPIs a surveiller

```

Listing 19 – Génération de la stratégie marketing

6.4 Pipeline d’enrichissement batch

```
1 # Avec Claude (Anthropic)
2 export LLM_PROVIDER=anthropic
3 export ANTHROPIC_API_KEY=sk-ant-...
4
5 python module5/enrichment_pipeline.py \
6     --input module2/output/products_scored.csv \
7     --output module5/output/ \
8     --max 50 # nb max de produits a resumer
9
10 # Sorties :
11 # module5/output/enriched_products.jsonl <- produits + resumes LLM
12 # module5/output/trend_report.md <- rapport de tendances
13 # module5/output/client_profile.md <- persona client
14 # module5/output/marketing_strategy.md <- plan marketing
15 # module5/output/enrichment_summary.json <- metriques
```

Listing 20 – Enrichissement batch complet

7 Module 6 — Architecture Responsable avec MCP

7.1 Concepts du Model Context Protocol

Model Context Protocol (MCP) : protocole standardisé développé par Anthropic pour encadrer les interactions des LLMs avec des outils et données externes, selon trois principes fondamentaux :

- 1. **Responsabilité** : chaque agent déclare ses intentions et ses sources
- 2. **Isolation** : les serveurs MCP n'exposent que le strict nécessaire
- 3. **Traçabilité** : journalisation complète de chaque interaction

7.2 Composants MCP implémentés

TABLE 9 – Composants de l'architecture MCP

Composant	Rôle	Outils exposés
MCP Host	Application Streamlit — initie les requêtes	N/A
MCP Client	Gateway unique — permission + rate-limit + routing	N/A
MCP Server : Data	Accès lecture seule aux données produits	get_top_products, get_cluster_profile, get_association_rules, get_market_stats
MCP Server : LLM	Exécution des chaînes LLM	summarize_product, generate_trend_report, build_client_profile, recommend_strategy, answer_question
MCP Server : Audit	Logs, permissions, rate limiting	Interne — non exposé

7.3 MCP Client — le gateway unique

Tout appel de l'application passe obligatoirement par le MCP Client, qui applique dans l'ordre :

- 1. **Vérification de routage** : l'outil existe-t-il ?
- 2. **Vérification de permission** : ce serveur a-t-il le droit d'appeler cet outil ?
- 3. **Vérification du rate limit** : la limite de fréquence est-elle respectée ?
- 4. **Exécution** : appel vers le serveur approprié
- 5. **Journalisation** : enregistrement dans l'audit JSONL

```
1 from module5.mcp.mcp_client import get_mcp_client
```

2

```

3 mcp = get_mcp_client()
4
5 # Appel securise vers le serveur Data
6 result = mcp.call("get_top_products", {"k": 10, "category": "
    Electronics"})
7 if result["allowed"]:
8     for produit in result["result"]:
9         print(f"{produit['title']} - score={produit['score']:.3f}")
10
11 # Appel securise vers le serveur LLM
12 result = mcp.call("answer_question", {
13     "question": "Quels produits Electronics recommandez-vous ?"
14 })
15 print(result["result"])
16 # -> "Sur la base des donnees actuelles, les 3 meilleurs..."
17
18 # Chaque appel genere une entree dans l'audit
19 # {"ts":"2025-...", "server":"data_server", "tool":
    get_top_products",
20 #   "allowed":true, "latency_ms":42.3, "result_size":1840}

```

Listing 21 – Utilisation du MCP Client

7.4 Matrice de permissions

TABLE 10 – Matrice de permissions par serveur

Serveur	Outils autorisés	Rate limit
data_server	get_top_products, get_cluster_profile, get_association_rules, get_market_stats	60 appels/min
llm_server	summarize_product, generate_trend_report, build_client_profile, recommend_strategy, answer_question	10 appels/min

Si data_server tente d'appeler summarize_product (qui appartient à llm_server), la requête est **automatiquement refusée** et journalisée avec allowed: false.

7.5 Journal d'audit

Chaque interaction est enregistrée dans module5/logs/audit.jsonl :

```

1 {"ts":"2025-06-01T14:23:11","server":"data_server",
2   "tool":"get_top_products","allowed":true,
3   "latency_ms":38.2,"result_size":2840}
4
5 {"ts":"2025-06-01T14:23:15","server":"data_server",
6   "tool":"summarize_product","allowed":false,

```

```
7  "reason":"Tool 'summarize_product' not permitted for 'data_server'
8  "}"
9  {"ts":"2025-06-01T14:23:20","server":"llm_server",
10  "tool":"generate_trend_report","allowed":true,
11  "latency_ms":2341.7,"result_size":1205}
```

Listing 22 – Format du journal d’audit MCP

7.6 Chatbot conversationnel intégré

Le chatbot Streamlit est accessible via la page “Assistant BI” du dashboard. Il route automatiquement les questions vers l’outil MCP approprié :

```
1  # "Quelles sont les tendances ?" -> generate_trend_report
2  # "Cree un profil client"         -> build_client_profile
3  # "Strategie marketing ?"     -> recommend_strategy
4  # "Top 5 produits ?"             -> get_top_products
5  # (toute autre question)        -> answer_question (RAG-lite)
```

Listing 23 – Routage intelligent des questions

8 Évaluation et Validation

8.1 Modèles supervisés

- **Séparation train/test 80/20** avec stratification sur `is_top_product` pour garantir la représentativité des classes dans les deux sous-ensembles
- **Validation croisée 5-fold** sur le train set pour détecter le surapprentissage
- **Métriques calculées** : accuracy, précision, rappel, F1-score, matrice de confusion
- **Justification du F1** : la classe positive représente $\approx 20\%$ du dataset ; l'accuracy serait trompeuse (un modèle prédisant toujours “négatif” aurait 80% d'accuracy)
- **Importances des features** exportées : les features `rating`, `log_review_count` et `log_price` sont systématiquement les plus discriminantes

8.2 Méthodes non supervisées

- **Méthode du coude** : courbe d'inertie pour KMeans sur $K \in [2, 10]$
- **Silhouette score** : coefficient entre -1 (mauvais) et $+1$ (excellent) ; valeur obtenue ≈ 0.186 (acceptable pour données e-commerce hétérogènes)
- **Validation visuelle** : projection PCA 2D des clusters pour vérifier la séparabilité
- **Interprétation métier** : chaque cluster est nommé et analysé selon ses profils moyens (section 3.4)

8.3 Règles d'association

- **Support minimum 5%** : garantit que la règle est observée sur au moins 5% des paniers
- **Confidence minimum 30%** : garantit une fiabilité statistique minimale
- **Lift > 1** : confirme que l'association est positive (au-delà du hasard)
- **Interprétation décisionnelle** : chaque règle est traduite en opportunité de cross-selling ou d'upselling concrète

9 Annexe — Captures d'écran du Dashboard BI

Cette annexe présente les captures d'écran de chacune des sept pages du dashboard **Smart eCommerce Intelligence**, accompagnées d'une description fonctionnelle. Le dashboard est accessible via la commande `streamlit run module4/app.py` et s'ouvre à l'adresse `http://localhost:8501`.

La barre latérale (sidebar) propose des **filtres globaux** — catégorie, plateforme, fourchette de prix — qui s'appliquent simultanément à toutes les pages, permettant une exploration cohérente du catalogue.

Page 1 — Overview (Vue d'ensemble)

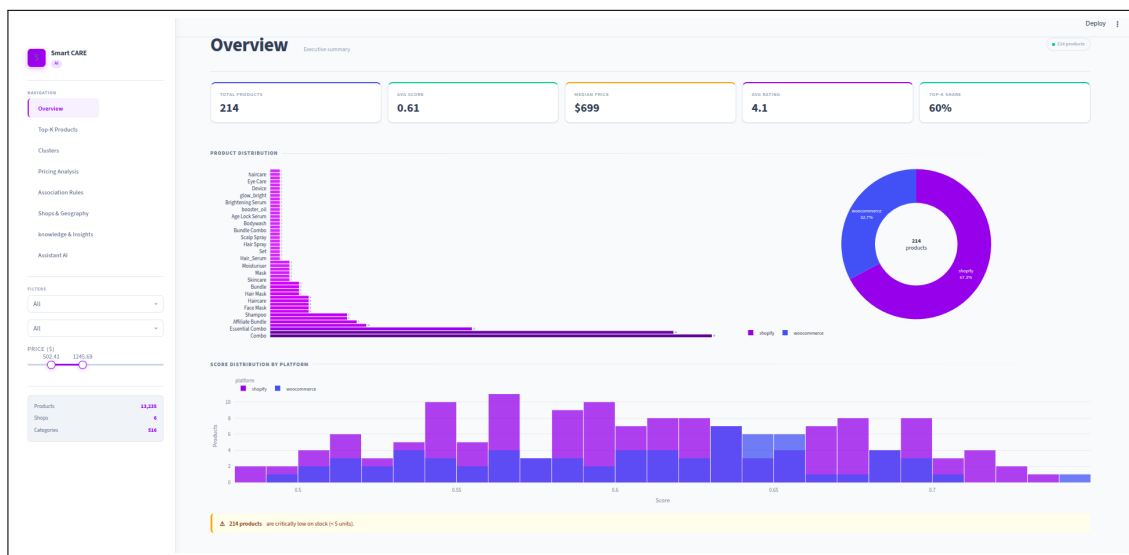


FIGURE 1 – Page Overview — KPIs et distributions globales du catalogue

Description : La page d'accueil affiche cinq **indicateurs-clés (KPIs)** sous forme de cartes colorées : nombre total de produits, score composite moyen, prix médian, note moyenne et pourcentage de produits “Top”. En dessous, trois visualisations Plotly permettent une lecture immédiate du catalogue : un histogramme horizontal des produits par catégorie, un camembert de répartition des plateformes (Shopify vs WooCommerce), et un histogramme de distribution des scores par plateforme. Cette page constitue le **point d'entrée décisionnel** du dashboard.

Page 2 — Top-K Produits

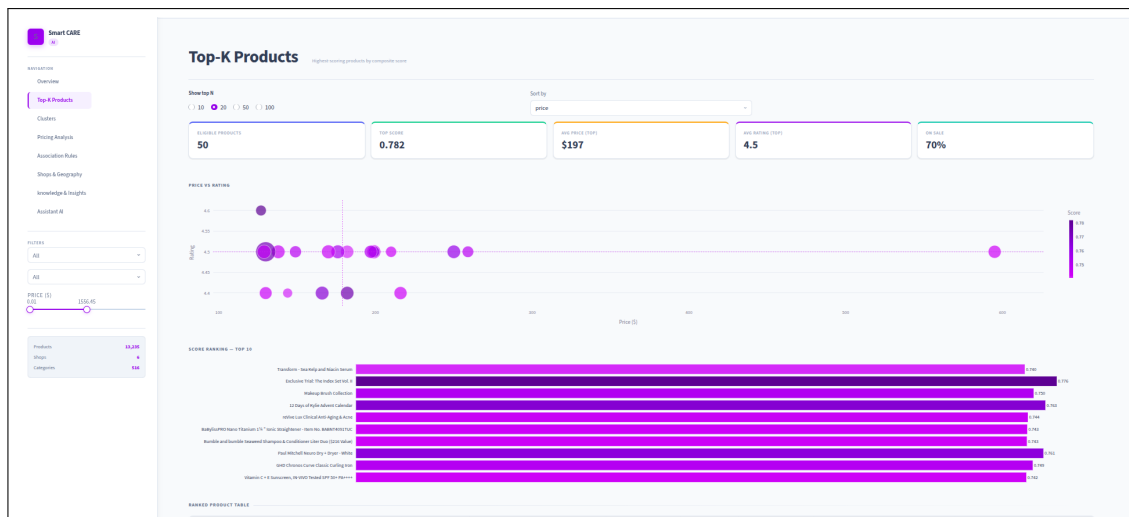


FIGURE 2 – Page Top-K — Scatter plot et tableau classé des meilleurs produits

Description : Cette page présente les produits les mieux classés selon le score composite. Un **scatter plot interactif** positionne chaque produit selon son prix (axe X) et sa note (axe Y) ; la taille des points est proportionnelle au nombre d'avis et la couleur au score (dégradé *viridis*). Le survol affiche le titre, la plateforme et la remise. En dessous, un **tableau stylisé** avec gradient de couleur sur la colonne score permet une lecture rapide du classement complet. Un curseur ajuste le nombre de produits affichés (10 à 100), et un **bouton d'export CSV** permet de télécharger la sélection.

Page 3 — Clusters (Segmentation produits)

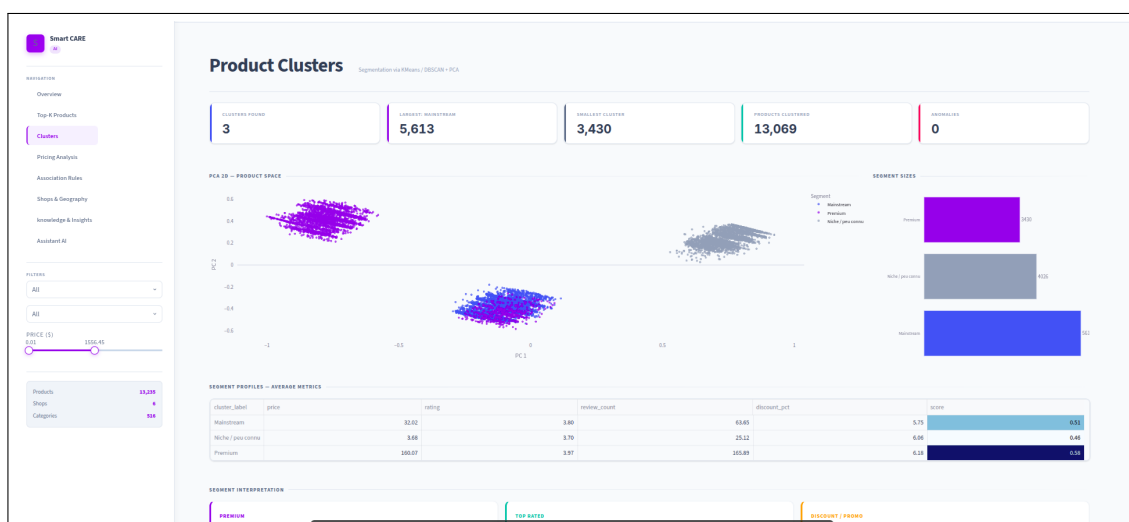


FIGURE 3 – Page Clusters — Projection PCA 2D et profils des segments

Description : La page de clustering visualise la **projection PCA 2D** du catalogue. Chaque point représente un produit, coloré selon son cluster : **Premium**, **Top rated**, **Discount/Promo**, Niche, **Mainstream**. Le graphe est entièrement interactif (zoom, survol,

filtrage par légende). À droite, un histogramme horizontal montre la taille de chaque segment. En bas, une **table de profils** présente les moyennes de prix, note, nombre d'avis, remise et score par cluster, avec un gradient de couleur YlGn pour identifier rapidement les segments les plus performants.

Page 4 — Analyse Prix



FIGURE 4 – Page Analyse Prix — Distribution des prix et des remises par catégorie

Description : Cette page explore la **structure tarifaire** du catalogue. Des **boxplots par catégorie** révèlent les médianes, quartiles et outliers de prix pour chaque segment. Un histogramme des remises (filtré sur les produits en promotion uniquement) montre la distribution des politiques de discount. Un scatter prix vs score permet de détecter les **produits sous-évalués** (score élevé, prix bas) ou surévalués. Trois métriques en bas de page synthétisent : nombre de produits en solde, remise moyenne et remise maximale.

Page 5 — Règles d'Association



FIGURE 5 – Page Règles d’association — Bubble chart support/confidence/lift

Description : La page des règles d’association présente les patterns de co-occurrence découverts par l’algorithme Apriori. Un **bubble chart** positionne chaque règle selon son support (axe X) et sa confiance (axe Y); la taille des bulles est proportionnelle au **lift** (force de l’association). Les règles à fort lift et forte confiance — identifiées dans le quadrant haut-droit — représentent les meilleures opportunités de **cross-selling**. Un curseur “min lift” filtre dynamiquement les règles affichées. Le tableau en dessous présente les règles complètes avec gradient de couleur sur la colonne lift. Trois métriques résument : nombre total de règles, lift maximum et confiance maximum.

Page 6 — Shops & Géographie

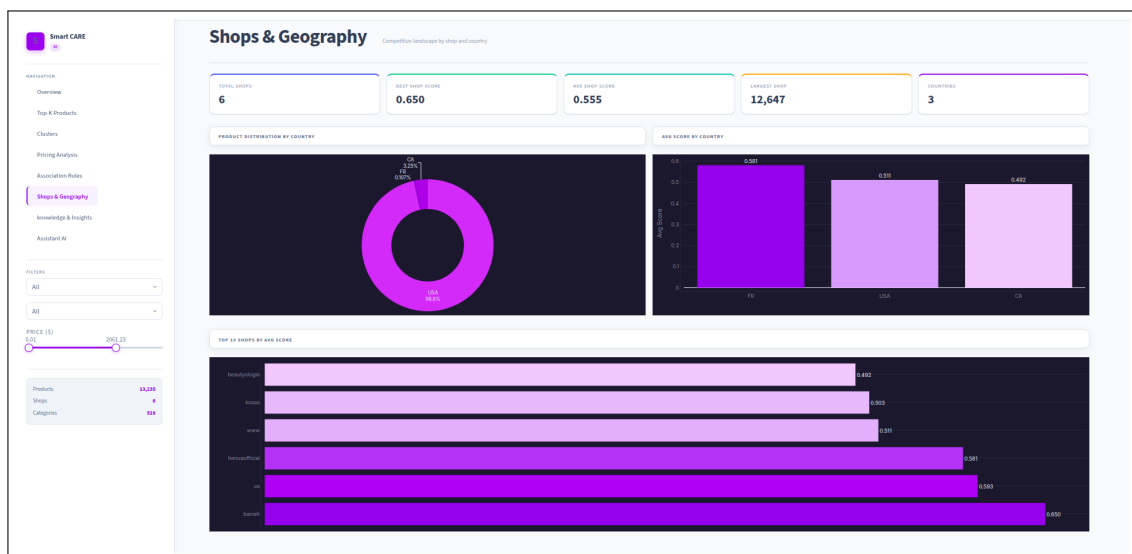


FIGURE 6 – Page Shops & Géographie — Classement des boutiques et répartition pays

Description : Cette page analyse la dimension **géographique et compétitive** du marché scrapé. Un histogramme horizontal classe les boutiques par score moyen décroissant, permettant d'identifier les **shops leaders**. Un camembert montre la répartition des produits par pays d'origine. Un troisième graphe en barres compare le score moyen par pays, révélant les marchés les plus performants. Enfin, un tableau détaillé liste toutes les boutiques avec leurs métriques (score moyen, nombre de produits, note moyenne) triées par score, avec gradient de couleur pour faciliter la lecture.

Page 7 — Assistant BI (Chatbot LLM)

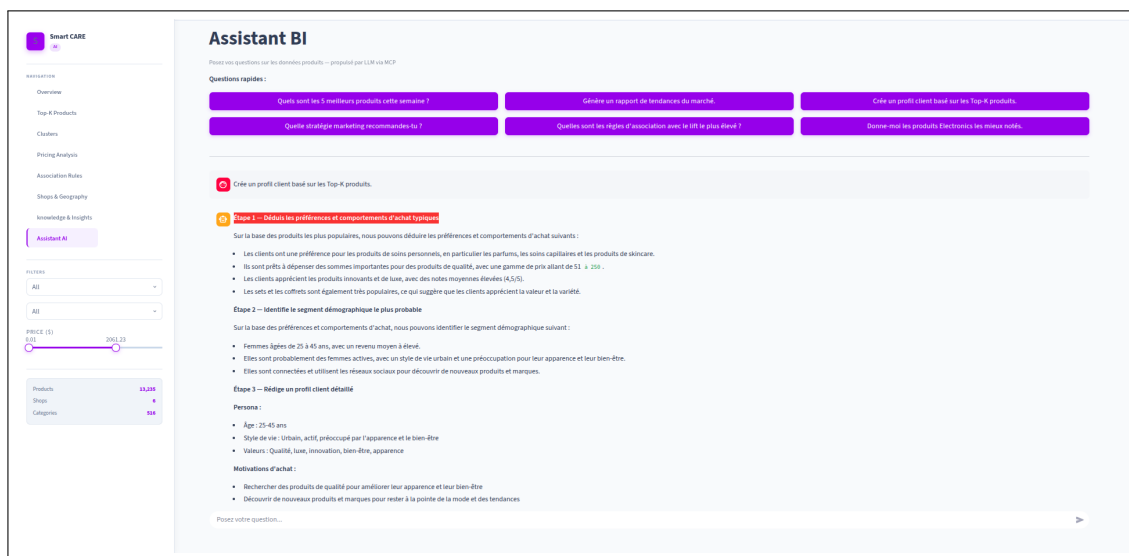


FIGURE 7 – Page Assistant BI — Interface conversationnelle pilotée par LLM et MCP

Description : La page Assistant BI offre une **interface conversationnelle en langage naturel**. Six boutons de questions rapides permettent d'interroger instantanément le système sur les tendances, le profil client, la stratégie marketing, les règles d'association, ou les Top-K produits. Le champ de saisie libre permet des questions personnalisées. En interne, chaque question est routée via le **MCP Client** vers le serveur approprié (Data ou LLM), avec vérification automatique des permissions et du rate limit. Un panneau extensible “Journal d’audit MCP” en bas de page affiche l'historique des appels (serveur, outil, latence, statut autorisé/refusé) pour la traçabilité complète des interactions.

TABLE 11 – Récapitulatif des pages du dashboard et leurs fonctions

#	Page	Visualisations principales	princi-	Valeur décisionnelle
1	Overview	KPI cards, barres, camembert, histogramme		Vue globale du catalogue
2	Top-K Produits	Scatter prix/note, tableau stylisé		Sélection des meilleurs produits
3	Clusters	PCA 2D interactif, profils clusters		Segmentation marché
4	Analyse Prix	Boxplots, histogramme remises, scatter		Stratégie tarifaire
5	Règles d'association	Bubble chart lift/confidence		Opportunités cross-selling
6	Shops & Géographie	Leaderboard, camembert pays		Analyse concurrentielle
7	Assistant BI	Chat LLM + audit MCP		Requêtes en langage naturel

Note aux lecteurs : Les captures d'écran présentées dans cette annexe ont été réalisées avec les données synthétiques intégrées au système (300 produits générés aléatoirement). Dans un contexte réel, ces visualisations reflèteront les données scrappées depuis les plateformes Shopify et WooCommerce cibles, via la commande `python orchestrator.py -urls <url_store>`.

10 Conclusion

Ce projet a permis de construire un système complet d'intelligence e-commerce en six modules couvrant l'intégralité du cahier des charges. Les principaux apports sont :

1. **Architecture A2A modulaire** : l'ajout d'une nouvelle plateforme e-commerce ne nécessite que l'implémentation d'une sous-classe **BaseAgent** (detect, scrape, clean) sans modifier l'orchestrateur.
2. **Pipeline ML reproductible** : chaque exécution via Kubeflow est versionnée, les artefacts (modèles, métriques, datasets) sont tracés dans l'artifact store, et le CI/CD garantit que les tests passent avant tout déploiement.
3. **Dashboard BI actionnable** : les six pages couvrent l'ensemble des angles d'analyse (performance, prix, clustering, géographie, règles), avec une page chatbot permettant des requêtes en langage naturel.
4. **Architecture LLM responsable** : le MCP garantit qu'aucun outil LLM n'accède à plus de données que nécessaire, avec un audit complet de chaque interaction et un rate limiting préventif.
5. **Robustesse demo** : les données synthétiques permettent de présenter et tester le système sans dépendance à un scraping réel ni à une clé API LLM.

Perspectives d'amélioration :

- Intégration de LLaMA en local (Ollama) pour supprimer la dépendance aux API externes
- Ajout d'une base vectorielle (FAISS, Chroma) pour un RAG plus riche dans le chatbot
- Monitoring en production via Prometheus + Grafana
- Extension à d'autres plateformes : Amazon, Etsy, AliExpress
- Détection de tendances temporelles avec analyse de séries chronologiques

Références

Références

- [1] Anthropic. (2024). *Model Context Protocol*. <https://modelcontextprotocol.io>
- [2] Anthropic. (2025). *MCP Specification 2025-03-26*. <https://modelcontextprotocol.io/specification/2025-03-26>
- [3] Kubeflow Community. (2024). *Kubeflow Pipelines Documentation*. <https://www.kubeflow.org/docs/components/pipelines/>
- [4] Google LLC. (2024). *kfp SDK Reference*. <https://kubeflow-pipelines.readthedocs.io/>
- [5] Shopify Inc. (2024). *Storefront API Reference*. <https://shopify.dev/docs/api/storefront>
- [6] WooCommerce. (2024). *REST API Documentation*. <https://woocommerce.github.io/woocommerce-rest-api-docs/>
- [7] Pedregosa, F. et al. (2011). Scikit-learn : Machine Learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- [8] Chen, T., & Guestrin, C. (2016). XGBoost : A Scalable Tree Boosting System. *Proceedings of KDD 2016*.
- [9] Agrawal, R., & Srikant, R. (1994). Fast algorithms for mining association rules. *Proceedings of VLDB 1994*.
- [10] Streamlit Inc. (2024). *Streamlit Documentation*. <https://docs.streamlit.io>
- [11] LangChain AI. (2024). *LangChain Documentation*. <https://python.langchain.com/>
- [12] Docker Inc. (2024). *Docker Documentation*. <https://docs.docker.com>
- [13] GitHub Inc. (2024). *GitHub Actions Documentation*. <https://docs.github.com/actions>